with a particular language and environment (and how willing they are to learn a new one!), what technology stack your project is using, and the goals and target audience of your BDD activities. Throughout the rest of this chapter, we'll compare and contrast several of the major BDD tools. The list of tools we'll study is far from exhaustive, but the techniques we'll discuss should be generally applicable no matter which tool you choose.

To explore the features of each tool, we'll look at how you might automate scenarios involving the Flying High airline's Frequent Flyer application, introduced in chapter 3. The Frequent Flyer program is designed to encourage flyers to fly with Flying High by allowing them to accumulate points that they can spend on flights or on other purchases. In this chapter's examples, we'll study some of the requirements around accumulating and managing Frequent Flyer points.

But first, let's take a general look at some of these tools' principles and uses.

6.2.1 Installing BDD tools

When it comes to installation, BDD tools fall generally into one of two categories. Some, like the original Ruby implementation of Cucumber, and most of the Cucumber family, are essentially command-line tools. These are easy to install and run from the command line, though they can be executed from a build script as well. For example, to run Cucumber scenarios written in a scripting language like Ruby, you simply place your feature files in a directory called features and run Cucumber from the command line.

Others are primarily run from within a build scripting environment (such as Maven, Ant, or Gradle in the JVM world), or from within an IDE. This includes JBehave (as you'll see in this section), Cucumber-JVM, and SpecFlow. These tools require a little more work and project infrastructure to set up.

6.2.2 Implementing step definitions

Implementing step definitions is at the heart of any BDD automation library. Step definitions tell the BDD tool exactly what code it should execute for each step in the feature file, and they extract any variables that are embedded in the text so that they can be used in the test code.

The way this is done is generally constrained by the language you're using to implement the steps. Many BDD tools use annotations, as illustrated here for JBehave:

```
@When("I register for the Frequent Flyer program")
public void whenIRegisterForTheFrequentFlyerProgram() {
  ...
}
```

Scripting languages such as Ruby and Groovy can take advantage of language features to make the step definitions more concise. In Ruby,[2] for example, a Cucumber step implementation might look like this:

[2] Although Ruby is a major actor in the BDD universe, I won't cover Cucumber in Ruby in as much detail as the other tools. This is mainly because of space—there are other great references for implementing BDD automation in Ruby, in particular *The Cucumber Book* by Matt Wynne and Aslak Hellesøy (Pragmatic Bookshelf, 2012).